

Tutorial3

Teaching Team of STA314

2026-09-24

Section 2: Gradient Descent

Quick Review: Gradient Descent with OLS

From our lecture last week, gradient descent is an **iterative optimization algorithm** used to minimize an objective function when:

- a **closed-form solution** is not available (e.g., logistic regression, neural networks),
- solving directly is **too expensive** (e.g., very large n and/or p), or
- the objective involves **non-smooth penalties** (e.g., the Lasso uses an L_1 penalty, which is not differentiable at 0, so we can use subgradients).

Suppose we want to solve

$$\hat{\beta} = \arg \min_{\beta \in \Theta} J(\beta),$$

where $J(\beta)$ is our objective function and Θ is the parameter space.

Starting from an initial guess $\beta^{(0)}$ (often all zeros), gradient descent updates

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \nabla J(\beta^{(t)}),$$

where $\nabla J(\beta)$ is the **gradient**, which points in the direction of steepest increase. Therefore, moving in the negative gradient direction decreases $J(\beta)$, provided the step size is suitable.

Learning rate α (step size):

- α too small: slow progress
- α too large: oscillations
- α much too large: instability or divergence

Stopping criteria (examples):

- $\|\nabla J(\beta^{(t)})\|$ is very small
- $\|\beta^{(t+1)} - \beta^{(t)}\|$ is very small
- reach a maximum number of iterations

Why use gradient descent even if a direct solution exists, as in linear regression?

For ordinary least squares,

$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$

However, computing and solving this system can become expensive when the number of predictors is very large. Gradient-based methods can instead update the parameters iteratively and can be adapted to stochastic or mini-batch settings.

Variants:

- **Batch GD:** uses all training observations for each update
- **SGD:** uses one observation for each update
- **Mini-batch GD:** uses a small batch of observations for each update

Section 2.1 Example Walkthrough

Example setup: Linear Regression + Gradient Descent

We will use a simple simulated linear regression example to illustrate gradient descent and see why the choice of learning rate matters.

For each $i = 1, \dots, n$, generate

$$x_i \sim N(0, 1),$$

and

$$y_i = 2 + 3x_i + \epsilon_i, \quad \epsilon_i \sim N(0, 1).$$

Therefore, the true regression coefficients are

$$\beta_0 = 2, \quad \beta_1 = 3.$$

Matrix notation

Let

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix},$$

and let X be the $n \times 2$ design matrix

$$X = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}.$$

Let

$$\boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix}.$$

Then the model can be written as

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\epsilon}.$$

Objective function

We fit linear regression by minimizing the sum of squared errors:

$$J(\beta) = \|\mathbf{y} - X\beta\|_2^2.$$

Equivalently,

$$J(\beta) = (\mathbf{y} - X\beta)^\top (\mathbf{y} - X\beta).$$

Some implementations use a scaled objective such as

$$\frac{1}{n} \|\mathbf{y} - X\beta\|_2^2.$$

Scaling does not change the minimizing value of β , but it does rescale the gradient and therefore changes what values of the learning rate α are reasonable.

Gradient of the objective

For

$$J(\beta) = \|\mathbf{y} - X\beta\|_2^2,$$

the gradient is

$$\nabla J(\beta) = 2X^\top (X\beta - \mathbf{y}).$$

For ordinary least squares, we could set

$$\nabla J(\beta) = 0$$

and solve directly for β . Instead, we will use gradient descent to illustrate the optimization procedure.

Gradient descent update rule

Gradient descent starts with some initial value $\beta^{(0)}$. Here, we initialize both coefficients at zero.

At each iteration,

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \nabla J(\beta^{(t)}).$$

For our least squares objective,

$$\beta^{(t+1)} = \beta^{(t)} - 2\alpha X^\top (X\beta^{(t)} - \mathbf{y}).$$

Why compare two learning rates?

The learning rate α controls how far we move at every iteration:

- α too small: slow progress
- α too large: the algorithm may overshoot the minimum
- α much too large: the algorithm may diverge

Before running gradient descent

- We know the true coefficients are $\beta_0 = 2$ and $\beta_1 = 3$. Do we expect the fitted coefficients to be exactly 2 and 3?
- If we initialize gradient descent at $(0,0)$, what do you expect the first update to do?
- Does the gradient descent know where the minimum is?

First, let us simulate the data.

```
set.seed(0)

n <- 200

x <- rnorm(n)

y <- 2 + 3 * x + rnorm(n, mean = 0, sd = 1)

# Design matrix: first column is the intercept
X <- cbind(1, x)

p <- ncol(X)

head(X)

##           x
## [1,] 1  1.2629543
## [2,] 1 -0.3262334
## [3,] 1  1.3297993
## [4,] 1  1.2724293
## [5,] 1  0.4146414
## [6,] 1 -1.5399500
```

Now define the sum of squared errors,

$$J(\beta) = \|y - X\beta\|_2^2,$$

and its gradient.

```
# Objective function:
# J(beta) = ||y - X beta||^2

sse <- function(beta) {

  residuals <- y - X %*% beta

  sum(residuals^2)
}

# Gradient:
```

```
grad_sse <- function(beta) {
  as.vector(
    2 * t(X) %*% (X %*% beta - y)
  )
}
```

Next, write a function that performs gradient descent for a specified learning rate.

```
run_gd <- function(alpha, T = 200, beta0 = rep(0, p)) {
  beta <- beta0
  fvals <- numeric(T)
  for (t in 1:T) {
    # Gradient descent update
    beta <- beta - alpha * grad_sse(beta)
    # Store the value of the objective function
    fvals[t] <- sse(beta)
  }
  list(
    beta = beta,
    fvals = fvals
  )
}
```

For comparison, we can fit ordinary least squares using `lm()`.

```
ols_fit <- lm(y ~ x)
beta_ols <- coef(ols_fit)
beta_ols
```

```
## (Intercept)          x
##    2.035452    3.020030
```

We should obtain estimates close to the true coefficients

$$\beta_0 = 2, \quad \beta_1 = 3.$$

Now compare two learning rates.

```
# A reasonably small learning rate
alpha_safe <- 0.002

# Reasonable learning rate
alpha_ok <- 0.9 * alpha_safe

# Learning rate that is too large
alpha_bad <- 10 * alpha_safe
```

```

# Run gradient descent
gd_ok <- run_gd(
  alpha = alpha_ok,
  T = 80
)

gd_bad <- run_gd(
  alpha = alpha_bad,
  T = 80
)

```

Reasonable learning rate

```

beta_ok <- gd_ok$beta
f_ok <- gd_ok$fvals

cat("Gradient descent with reasonable learning rate\n")

## Gradient descent with reasonable learning rate
cat("alpha =", alpha_ok, "\n\n")

## alpha = 0.0018
cat("Final beta:\n")

## Final beta:
print(round(beta_ok, 6))

## [1] 2.035452 3.020030
cat("\nOLS beta:\n")

##
## OLS beta:
print(round(beta_ols, 6))

## (Intercept)          x
##    2.035452    3.020030
cat("\nSSE at final beta:", round(sse(beta_ok), 6), "\n")

##
## SSE at final beta: 198.2599
cat(
  "Distance between GD and OLS estimates:",
  sqrt(sum((beta_ok - beta_ols)^2)),
  "\n"
)

## Distance between GD and OLS estimates: 3.140185e-15
cat("\nFirst 5 SSE values:\n")

##
## First 5 SSE values:

```

```
print(round(head(f_ok, 5), 4))

## [1] 506.8982 241.3994 204.5215 199.1885 198.3992
cat("\nLast 5 SSE values:\n")

##
## Last 5 SSE values:
print(round(tail(f_ok, 5), 4))

## [1] 198.2599 198.2599 198.2599 198.2599 198.2599
```

Discussion

- What evidence in the output tells us that the gradient descent is converging?
- Why does the gradient descent estimate eventually agree with the OLS estimate?
- When should we stop iterating?

Next...

- What do you predict will happen with a larger learning rate?
- When would a larger learning rate be helpful?

Learning rate that is too large

```
beta_bad <- gd_bad$beta
f_bad <- gd_bad$fvals

cat("Gradient descent with too-large learning rate\n")

## Gradient descent with too-large learning rate
cat("alpha =", alpha_bad, "\n\n")

## alpha = 0.02
cat("Final beta:\n")

## Final beta:
print(beta_bad)

## [1] -7.886447e+67  5.916887e+66
cat("\nFirst 10 SSE values:\n")

##
## First 10 SSE values:
print(head(f_bad, 10))

## [1] 8.885800e+04 3.463813e+06 1.403424e+08 5.897770e+09 2.564003e+11
## [6] 1.148053e+13 5.266424e+14 2.461976e+16 1.167431e+18 5.593648e+19
```

With this much larger learning rate, the parameter updates repeatedly overshoot the minimum. Rather than decreasing, the objective function becomes larger and larger.

Plotting the optimization path

Since the SSE for the unstable learning rate becomes extremely large, it is useful to plot the SSE on a $\log_{10}$ scale.

```

iterations <- seq_along(f_ok)

ylim <- range(
  c(log10(f_ok), log10(f_bad)),
  finite = TRUE
)

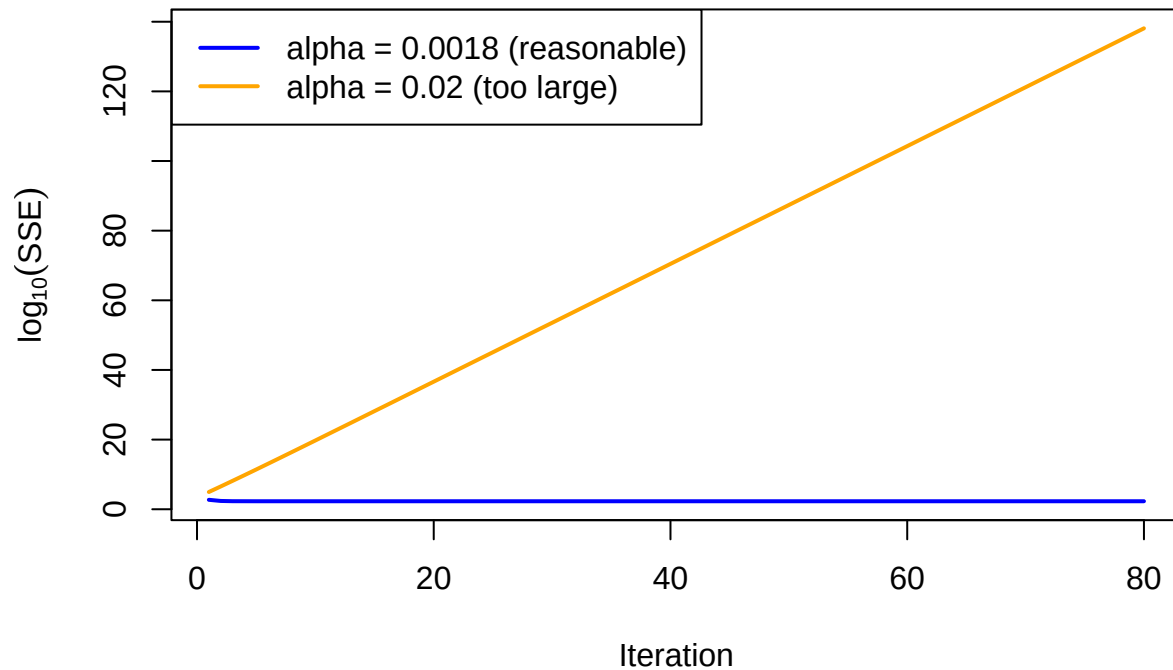
plot(
  iterations,
  log10(f_ok),
  type = "l",
  lwd = 2,
  col = "blue",
  ylim = ylim,
  xlab = "Iteration",
  ylab = expression(log[10](SSE)),
  main = "Gradient Descent Training Curve"
)

lines(
  iterations,
  log10(f_bad),
  lwd = 2,
  col = "orange"
)

legend(
  "topleft",
  legend = c(
    paste0("alpha = ", alpha_ok, " (reasonable)"),
    paste0("alpha = ", alpha_bad, " (too large)")
  ),
  col = c("blue", "orange"),
  lty = 1,
  lwd = 2
)

```


Gradient Descent Training Curve



We could also plot the MSE or RMSE rather than the SSE. The overall pattern would be the same.

Extra Discussion

1. Looking at the two runs above, what is the trade-off when choosing the learning rate α ?
2. Suppose the algorithm is converging, but extremely slowly. What would you try changing?
3. Suppose the objective function starts increasing rapidly. What would you try changing?
4. Would changing the starting value of β fix a learning rate that is much too large?
5. At convergence, what should be approximately true about the gradient?
6. How could we choose an appropriate learning rate given the data?

Hint: Think about the second derivative